ことのは：日本語の文章として読めるアクター並列言語の設計と実現

—— 一文方式による曖昧さのない文法と，二つの表記の相互変換 ——

児玉 靖司
法政大学

2026 年 11 月

概要

日本語の文章としてそのまま読め，しかも読み方がひと通りに決まるプログラミング言語「ことのは」を設計し，実装した。計算の単位はアクターであり，プログラムは「哲学者が『渡す』というメッセージを受け取ったとき，……を送る。」のような文の並びとして書く。字下げを使わない**文章形式**と，字下げで構造を示す**構造化形式**の二つの表記を持ち，両者を機械的に相互変換できる。

自然な日本語に見せながら読み方を一意にするため，「〜ならば,」などの本体を**その一文だけ**に限り，二文以上は「次のことを行う。…以上。」で囲む**一文方式**を採った。この文法を bison で検査すると衝突は 0 件（90 規則・185 状態）である。条件を直接入れ子にできる版では衝突がちょうど 1 件（else の付き先）生じ，これがシフト優先で解消できることも示した。また，アクター定義の本体を入れ子の規則で書くと衝突が 26 件生じ，平らな並びにして意味段階で振り分けると 0 件になることを確かめた。

ブラウザ上で動く処理系（構文解析・逆変換・実行）を同じ文法ファイルから生成した。C/Python 版との出力は見本 10 本と変異 300 本のすべてで一致した。5 人の哲学者問題では，番号の小さいフォークから取る版は乱数の種 1〜1000 のすべてで完走し，全員が左から取る版は 721 回でデッドロックした。ユークリッドの互除法・ハノイの塔・バブルソート・クイックソートの 4 例題も，種 1〜300 のすべてで正しい答えを返した。

1　はじめに

プログラムを母語の文章として書きたいという要求は古い。COBOL は英文に似せた構文を持ち，日本では「なでしこ」[9] や「ドリトル」[10] が日本語の語彙でプログラムを書けるようにしてきた。Inform 7 [11] は英語の文章そのものをプログラムとして扱う。一方で，自然言語に近づくほど**同じ文章が二通りに読める**危険が増す。日本語には語の区切りの空白がなく，「〜ならば,」の後ろのどこまでが条件の本体かも文面からは決まらない。

本研究の問いは次の二つである。

1. 日本語の文章として不自然でない書き方のまま，読み方が**ひと通りに決まる**文法を作れるか。
2. その文法で，**並列に動く**プログラム（アクター [1, 2]）を書き，実際に動かせるか。

これに答えるため，日本語アクター言語「ことのは」を設計し，文法を bison [7] で検査し，ブラウザ上で動く処理系を作った。本稿の貢献は次のとおりである。

- 本体の範囲を「一文」に限る**一文方式**により，字下げも括弧も使わずに衝突 0 の LR(1) [5] 文法を得たこと（5 節）。
- 字下げのない**文章形式**と，字下げで構造を示す**構造化形式**を定め，往復しても構造が一字も変わらない変換器を作ったこと（7 節）。
- 同じ文法ファイルからブラウザ用の解析器を生成し，アクターの実行系をつけて，並列の非決定性とデッドロックを観察できるようにしたこと（8 節）。
- 文法の衝突・変換の一致・実行結果を，すべて道具で測ったこと（9 節）。止まらなかった誤りも記録した。

処理系と見本は `https://kodamay.org/kotonoha/` で公開しており，ブラウザだけで試せる。

2　関連研究

日本語で書く言語．なでしこ [9] は助詞で引数の役割を示し，語順の自由度を持たせている。ドリトル [10] は教育用のオブジェクト指向言語で，「タートル！ 100 歩く。」のように日本語の語順でメッセージを送る。これらは語彙を日本語にしているが，制御構造の範囲は「ここまで」などの終端語や字下げで示すことが多い。ことのはは，範囲を**文の切れ目そのもの**で示す点が異なる。

字下げと文法．Landin の off-side rule [6] 以来，字下げでブロックを示す言語は多いが，字下げは文脈自由文法の外で字句解析器が処理する。ことのはの文章形式は字下げを持たず，全体を一つの文脈自由文法で書ける。

アクター．Hewitt らのアクター [1] と Agha の定式化 [2] では，計算の単位は状態を共有せずメッセージで通信するアクターである。Erlang [3] はこれを実用言語にした。著者らはアクターに基づく並列言語 AIPL（Actor based Intelligent Parallel Language）[12] も開発している。ことのはは同じ計算モデルを，日本語の文章で書けるようにする試みである。

3　設計方針

設計目標を次の五つに定めた。

G1 **文章として読める**　句読点「、」「。」，連用形と終止形，「〜を受け取ったとき、」「〜ならば、」といった普通の日本語の言い回しで構造を表す。字下げや記号の括弧に頼らない。

G2 **読み方が一つ**　正しい文章なら構文木はひと通りに決まる。これを人の目ではなく，構文解析器生成系の衝突検査で保証する。

G3 **二つの表記**　文章形式に加えて，字下げで構造を見せる構造化形式を持ち，どちらからでも他方を機械的に作れる。

G4 **並列**　計算の単位はアクター。状態は共有せず，メッセージを受け取ったときの動作として書く。

G5 **試せる**　インストールなしに，ブラウザだけで書いて動かせる。

G1 と G2 は互いに引っ張り合う。自然な日本語ほど省略と照応（「その哲学者」）が多く，読み方が揺れる。本研究の設計判断の多くは，この二つの折り合いをどこで付けるかの選択である。

4　言語の概要

プログラムは，アクター定義の並びと，「実行を始める。」から「これで実行を終わる。」までの実行部からなる。アクター定義は「X というアクターを定義する。」で始まり，状態の宣言（「X は A と B を持つ。」），初期値の文，受信節を経て，「これで X の定義を終わる。」で閉じる。受信節は「X が『m』というメッセージ［と引数］を受け取ったとき、」で始まり，その本体は次の受信節か定義の終わりまで続く。

図 1 に，5 人の哲学者問題 [4] のフォーク側の定義を示す。フォークはアクターであり，使用中なら依頼してきた哲学者を待ち行列に入れ，返されたら待ち行列の先頭に渡す。

文の種類を表 1 にまとめる。一つの文の中では，動作を連用形（「変更し、」「送り、」）でつなぎ，最後を終止形（「変更する。」「送る。」）で閉じる。これは日本語の「〜し、〜し、〜する。」という普通の連ね方そのものであり，同時に**文の終わりを字句の上で確定させる**働きをする。

値は数・文字列・真偽（はい／いいえ）・なし・配列・アクターである。配列は「列の（現在の番号）番目」で読み書きし，「列の要素数」で長さを得る。計算は必ず括弧で囲む（「(A に B を加えた数)」「(A から B を引いた数)」「(A を B で割った余り)」）。括弧を義務にしたのは，「A から B を引いた数に 2 を加える」のような文で，助詞「に」が計算の一部なのか加算文の一部なのかを決められなくなるからである。

文章形式（字下げなし）

フォークというアクターを定義する。
フォークは番号と使用中と待ち行列を持つ。
番号の初期値は 0 とし、使用中の初期値はいいえとし、待ち行列の初期値は空とする。
フォークが「取りたい」というメッセージと哲学者を受け取ったとき、使用中がいいえならば、使用中をはいに変更し、その哲学者に「渡す」というメッセージを送る。そうでなければ、その哲学者を待ち行列の末尾に追加する。
フォークが「返す」というメッセージを受け取ったとき、待ち行列が空でなければ、待ち行列の先頭から次の哲学者を取り出し、次の哲学者に「渡す」というメッセージを送る。そうでなければ、使用中をいいえに変更する。
これでフォークの定義を終わる。

構造化形式（解析器の出力）

```
「フォーク」というアクターを定義する。
    状態は 番号、使用中、待ち行列。
    初期状態は
        番号 は 0。
        使用中 は いいえ。
        待ち行列 は 空。
    以上。
    「取りたい」と 哲学者 を受け取ったとき、
        もし 使用中 が いいえ ならば、
            使用中 は はい。
            哲学者 に「渡す」を送る。
        そうでなければ、
            待ち行列 に 哲学者 を追加する。
        以上。
    以上。
    「返す」を受け取ったとき、
        もし 待ち行列 が 空 でない ならば、
            次の哲学者 は 待ち行列 の先頭を取り出す。
            次の哲学者 に「渡す」を送る。
        そうでなければ、
            使用中 は いいえ。
        以上。
    以上。
定義を終わる。
```

図 1　5 人の哲学者問題のフォーク。上の文章形式を解析すると下の構造化形式が出る。下から上も機械的に作れる。

5　文法：一文方式

5.1　本体の範囲をどう決めるか

字下げのない日本語で最も難しいのは，「〜ならば、」の本体がどこまでかである。次の文章を考える。

> 値が 1 ならば、「一」という文字列を表示する。「二」という文字列を表示する。

二文目が条件の内側か外側かは，日本語としてはどちらにも読める。ことのはは，これを**一文方式**で決める。

- 「〜ならば、」「そうでなければ、」「〜について、」の本体は，**その一文だけ**である。
- 二文以上を入れるときは「次のことを行う。」と「以上。」で囲む。
- 厳格版では，一文の本体に条件文を直接は書けない。入れ子は「次のことを行う。」の中に書く。

したがって上の例の二文目は条件の外側にある。構造化形式に変換すると，それが字下げとして目に見える（9.2 節）。「次のことを行う。…以上。」は他言語の `{ }` にあたるが，日本語の文章として読んで不自然でない言い回し

表 1　文の種類（抜粋）。構造化形式は解析器が出す形。

種類	文章形式の例	構造化形式
表示	回数と「回目」という文字列を表示する	**回数 と「回目」を表示する**
送信	先のフォークに「取りたい」というメッセージと自分自身を送る	**先のフォーク に「取りたい」と 自分 を送る**
一斉送信	哲学者たちの全員に「開始」というメッセージを送る	**哲学者たち の全員に「開始」を送る**
代入	使用中をはいに変更する／人の番号を 3 に設定する	**使用中 は はい**
加算	食べた回数に 1 を加える	**食べた回数 は 食べた回数 足す 1**
追加	その哲学者を待ち行列の末尾に追加する	**待ち行列 に 哲学者 を追加する**
取り出し	待ち行列の先頭から次の哲学者を取り出す	**次の哲学者 は 待ち行列 の先頭を取り出す**
生成	フォークというアクターを 1 つ生成し、棒と名付ける	**棒 は「フォーク」の新しいアクター**
条件	持っている本数が 2 ならば、…	**もし 持っている本数 が 2 ならば、**
繰り返し	1 から 5 までの各番号について、…	**1 から 5 まで繰り返す。**
計算	（枚数から 1 を引いた数）／（a を b で割った余り）	**（枚数 引く 1）**

表 2　bison 3.8.2 による文法の検査

文法	規則	状態	衝突
厳格版（標準）	90	185	0
許容版（条件を直接入れ子にできる）	91	187	S/R 1
厳格版の定義本体を入れ子の規則で書いた版（比較用）	—	—	S/R 26

を選んだ。

5.2　文法の骨格

文法は 90 規則，終端記号 70 種，非終端記号 34 種からなる。骨格を次に示す（抜粋。全体は公開ページに掲載）。

BNF（抜粋，bison の報告書から自動生成）

```
<文>          ::= <一文の中身> "。" | "その後、" <一文の中身> "。" | <条件文> | <繰り返し文>
<一文の中身>  ::= <終止節> | <連用節> "、" <一文の中身>
<条件文>      ::= <条件> <本体> | <条件> <本体> "そうでなければ、" <本体>
<本体>        ::= <一文の中身> "。" | "次のことを行う。" <文の並び> "以上。"
<条件>        ::= <式> "が" <式> "ならば、" | <式> "が" <式> "でなければ、"
                  | <式> "が" <式> "より小さければ、"
<繰り返し文>  ::= <式> "から" <式> "までの各番号について、" <本体>
<終止節>      ::= <表示の前半> "表示する" | <送信の前半> "送る" | <変更の前半> "変更する|設定する" | …
<連用節>      ::= <表示の前半> "表示し"   | <送信の前半> "送り" | <変更の前半> "変更し|設定し"   | …
<計算>        ::= "（" <式> "に" <式> "を" "加えた数" "）" | "（" <式> "から" <式> "を" "引いた数" "）"
                  | "（" <式> "を" <式> "で割った余り" "）"
```

終止節と連用節は，同じ「前半」に異なる語尾が付いた形である。語尾の違いだけで「文がここで終わるか」が決まるので，解析器は 1 字句の先読みで判断できる。

5.3　検査結果

bison 3.8.2 で検査した結果を表 2 に示す。厳格版は衝突 0 件である。

■定義本体の平坦化． アクター定義の本体は，素直に書けば「初期値の文の並び，続いて『受信節の頭＋文の並び』の並び」である。しかしこう書くと衝突が 26 件生じる。受信節の頭「哲学者が『渡す』というメッセージを受け取ったとき、」と，条件文「哲学者が……ならば、」は，どちらも〈式〉「が」で始まる。解析器は〈式〉を読み始める前に「前の受信節の文の並びをここで閉じるか」を決めねばならないが，その答えは〈式〉と「が」の後ろまで読まないと分からない。〈式〉は「人の先のフォーク」のようにいくらでも長くなるので，どんな k でも LR(k)

では決められない。

そこで本体を，初期値の文・受信節の頭＋一文・文，の三種の項目の**平らな並び**として受け取り，どの文がどの受信節に属するかは，解析後の意味の段階で振り分けることにした。これで衝突は 0 件になる。振り分けの段階では，「初期値の文が受信節より後ろにある」「受信節の外に文がある」を意味エラーとして報告する。これは文法の表現力を落とさずに，判断を「見れば分かる」位置まで遅らせる方法である。

■許容版と else の付き先． 本体に条件文を直接書けるようにした許容版では，衝突がちょうど 1 件生じる。bison の反例は次のとおりである。

bison -Wcounterexamples の出力（抜粋）

```
shift/reduce conflict on token SOUDENAKEREBA
  Example: cond cond part •  SOUDENAKEREBA part
```

「A ならば B ならば C。そうでなければ D。」の「そうでなければ」が，内側と外側のどちらの「ならば」に付くかという，C 言語などと同じ dangling else である。シフトを優先すると「一番近い『ならば』に付く」と一意に決まるので，`%expect 1` を明記して受け入れることはできる。しかし日本語の読み手がそう読むとは限らないため，標準には厳格版を採った。許容版は，曖昧さが**どこに一件だけ**残るかを示す比較対象として残している。

6　字句解析：名前を宣言から切り出す

日本語には語の区切りの空白がないので，「先のフォークに」をどこで切るかを字句解析器が決めねばならない。ことのはでは，名前を**宣言された場所**から集める。アクター名（「〜というアクターを定義する」），状態（「〜は…を持つ」），受信の引数（「〜というメッセージと〜を受け取ったとき」），「〜と名付ける」「〜の先頭から〜を取り出す」の五か所である。字句解析器はまず文章全体からこれらを集め，次に予約語（約 70 語）と集めた名前を**長い順**に照合して，最初に一致したものを取る（最長一致）。

最長一致は単純だが，予約語の切り方を誤ると静かに壊れる。開発中，「使用中の初期値はいいえ」が「使用中」「の」「初期値」「は」「いいえ」ではなく，「…初期値」「はい」「いえ」と切れる不具合があった。「はい」が予約語として「は」より長いためである。「の初期値は」を一つの予約語にすることで解消した。このように，予約語の一覧は文法と一体で設計する必要がある。

名前を正規表現で前もって集める方式には限界もある。宣言の言い回しが崩れていると名前が集まらず，誤りは「予約語でも宣言された名前でもない」という字句エラーとして，使った場所で報告される（表 4 の E3）。

7　二つの表記と相互変換

7.1　構造化形式

構造化形式は，ブロックを字下げで見せ，同時に「以上。」「定義を終わる。」「実行を終わる。」で閉じる（図 1 下）。閉じる語があるので，字下げを見なくても行単位で読める。字下げは人のためであり，機械は使わない。

7.2　変換器

文章形式から構造化形式への変換は，構文解析の動作として行う。bison の各規則の動作が構造化形式の文字列を直接組み立て，構文木を経由しない。逆向き（構造化形式から文章形式）は，行単位で読んで木にし，文章を書き出す（Python で 244 行）。

逆変換が出すのは**標準の書き方**である。元の言い回しの一部は構文木に残らないため，元の文章には戻らない。

- 「その哲学者」の「その」は意味を持たないので消える。
- 単純な文が続くところは，3 つまでを「、」でつないで一文にする。

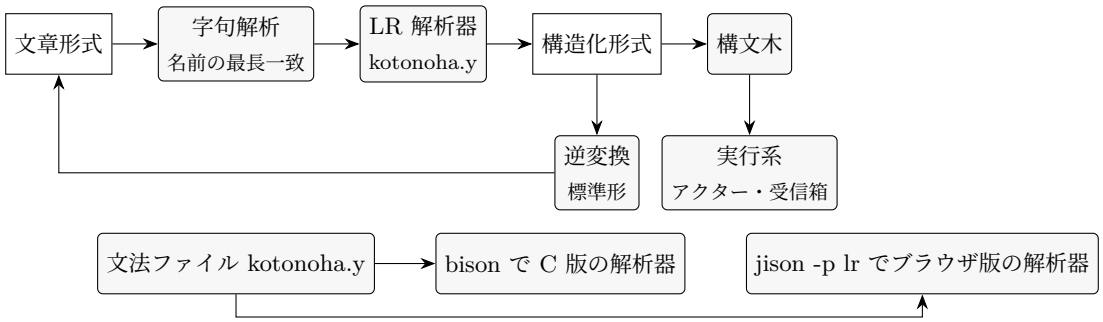


図 2　処理系の構成。一つの文法ファイルから C 版とブラウザ版の解析器を作る。実行系は構造化形式から作った構文木を解釈する。

- 代入は，対象が「人の番号」のように「の」を含めば「設定する」，含まなければ「変更する」にする。

このため，変換の正しさを「元に戻ること」では測れない。代わりに次の二条件を合格の条件とした。

1. 文章 $\to$ 構造 S_1 $\to$ 文章 $'$ $\to$ 構造 S_2 としたとき，$S_1 = S_2$（一字一句）。
2. 文章 $'$ をもう一度往復させても変わらない（標準形は不動点）。

条件 1 は，逆変換が意味を変えていないことを，条件 2 は標準形が安定していることを示す。

7.3　処理系の構成

処理系の構成を図 2 に示す。ブラウザ用の解析器は，bison に渡す文法ファイルそのものを jison [8]（bison 互換の JavaScript 用生成器）の形式に機械的に写して生成する。規則の動作は C と JavaScript で同じ式になるように書いてあるので，そのまま写せる。手で別の解析器を書かないので，文法がずれることがない。なお jison の LALR モードは，bison では衝突しない箇所（代入と追加の前半が同じ形になる所）に偽の衝突を報告した。正準 LR モード（`-p lr`）では衝突はない。

8　実行系

実行系はブラウザ上の JavaScript で書いた（実行部を含めて 565 行）。文章形式を構造化形式に変換し，逆変換器と同じ読み取り部で構文木にし，それを解釈する。逆変換器の木を使い回すので，どちらの表記で書いたプログラムも同じように動く。

■意味． 各アクターは受信箱を一つ持ち，届いたメッセージを**一通ずつ**処理する。一つのアクターの中は逐次であり，アクター間で状態は共有しない。送信は非同期で，送った側は待たない。受信箱は届いた順（FIFO）に処理される。実行部（「実行を始める。」の中）は「自分」を持たない特別な主体が上から一度だけ実行する。ここでは，作ったアクターの状態に直接値を入れられる（「人の番号を現在の番号に設定する」）。初期配置を書くための例外である。繰り返しの範囲は入る前に一度だけ計算するので，入れ子の内側の範囲には外側の「現在の番号」が使える。

■動く順． 次にどのアクターが一通を処理するかは，「送られた順」（FIFO）か，種つき乱数（mulberry32）で選ぶ。乱数の種を固定すれば同じ動きを何度でも再現でき，種を変えれば並列ならではの別の動きになる。並列の非決定性を，教室で**再現できる形で**見せるための仕組みである。

■止まったときの報告． すべての受信箱が空になると実行は終わる。その時点で，状態の中に**アクターだけが並ぶ配列**（待ち行列）があり，中にアクターが残っていれば，「待たされている」と報告する。デッドロックでは，これが待ちの輪として現れる。当初は「アクターを含む配列」を数えていたが，ハノイの塔の例題で依頼の組（数・文字列・自分自身を並べた配列）を待ちと誤報したため，アクターだけからなる配列に限った。

表 3　変換の検査

検査	結果
見本 10 本の往復（条件 1：$S_1 = S_2$）	10/10
同（条件 2：標準形の不動点）	10/10
C/Python 版とブラウザ版の出力の一致（見本 10 本，違反例 1 本を含む）	10/10
変異 300 本の判定と出力の一致	300/300（受理 103，拒否 197）

表 4　壊したプログラムへの反応（ブラウザ版の表示をそのまま引用）

	壊し方	反応
E1	条件を直接入れ子にする	構文エラー 5 行:「が」の所で止まった。ここに来られるのは: と、を、に、…
E2	「次のことを行う。」の「以上。」を忘れる	構文エラー 8 行:「これで」の所で止まった
E3	宣言していない名前を使う	字句エラー 5 行:「数が 1 ならば…」は予約語でも宣言された名前でもない
E4	文を連用形で閉じる（「表示し。」）	構文エラー 6 行:「。」の所で止まった。ここに来られるのは: 、
E5	定義を別の名前で閉じる	意味エラー:「係」の定義を「値」で閉じている
E6	受信節のないメッセージを送る	実行時エラー: 係#1 は「終わり」を受け取る受信節を持たない
E7	空の配列の 3 番目を読む	実行時エラー:「列の 3 番目」: 3 番目は無い（0 個しかない）
E8	0 で割った余り	実行時エラー: 0 で割ろうとした
E9	文字列に数を加える	実行時エラー: 二 は数ではない
—	条件の後ろに二文目を続ける	**止まらない**。二文目は条件の外になる

9　評価

測定はすべて 2026 年 11 月 11 日に，公開版と同じ処理系で行った。

9.1　変換の正しさ

結果を表 3 に示す。変異は，哲学者の文章から文を消す・入れ替える・重ねるなどして作った 300 本である。C/Python 版とブラウザ版は，受理・拒否の判定と出力のすべてで一致した。哲学者のプログラム（空白を除いて 1,299 字，379 字句）の変換にかかる時間は，ブラウザ版で 1 回あたり 0.59 ms であった。

9.2　誤りへの反応

正しい小さなプログラムを 1 か所ずつ壊し，処理系の反応を記録した（表 4）。九通りのうち，文法の誤り（E1〜E5）は解析の段階で止まり，計算の誤り（E6〜E9）は実行時エラーとして止まった。

二つの点に注意が要る。第一に E2 では，原因（5 行目の「次のことを行う。」）から離れた 8 行目で止まった。閉じ忘れは，閉じる語を待っている間に次の定義の頭が来て初めて分かる。括弧の閉じ忘れと同じ性質であり，報告位置を原因に近づけるのは今後の課題である。

第二に，最後の行の「値が 1 ならば、『一』を表示する。『二』を表示する。」は**誤りとして止まらない**。一文方式では二文目は条件の外側であり，文法上は正しいからである。書き手が両方を内側のつもりで書いた場合，その意図とのずれは文法では検出できない。ことのはでは，このずれを**構造化形式で見せる**ことで補う。この例の構造化形式では「『二』を表示する。」が「以上。」の後ろ，一段浅い位置に出るので，条件の外にあることが目で確かめられる。二つの表記を持つことの実用上の意味はここにある。

表 5　5 人の哲学者（乱数の種 1〜1000）

版	完走	停止	処理したメッセージ数
安全版（小さい番号から取る）	1000	0	どの種でも 105 通
危険版（全員が左から取る）	279	721	最小 20，中央値 27，最大 105

表 6　例題（乱数の種 1〜300）

例題	行	アクター	メッセージ	出力の並び	答え
ユークリッドの互除法（2 組を並行に）	28	2	12	202 通り	21 と 2
ハノイの塔（4 枚）	39	9	30	1 通り	15 手
バブルソート	23	1	1	1 通り	1〜9 の昇順
クイックソート（左右を並列に）	50	12	22	56 通り	1〜9 の昇順

9.3　5 人の哲学者

5 人の哲学者問題 [4] を二通りに書いて，乱数の種 1〜1000 で実行した（表 5）。安全版は，各哲学者が**番号の小さいフォークから**取る（哲学者 5 だけはフォーク 1 を先に取る）。資源に順序を付けて待ちの輪を作らせない古典的な方法である。危険版は，全員が左のフォークから取る。各哲学者は 3 回食べると満腹になって止まる。

危険版が止まった 721 回について，止まるまでに何回の食事があったかを数えると，0 回が 459，1 回が 95，2 回が 34 で，以後 3〜10 回まで 26 回以下ずつ続いた。3 分の 2 近くは一度も食べないうちに全員が左を握って止まるが，何回か食べてから止まる場合も 262 回ある。デッドロックは「最初に起きなければ安全」ではない。これは回数だけを数えると見えない。停止時には，たとえば「フォーク#1 の待ち行列で哲学者#5 が待っている／フォーク#2 の待ち行列で哲学者#1 が待っている／…」と，5 人の待ちの輪が表示される。送られた順（FIFO）で動かすと，安全版は完走し，危険版は 20 通目で止まる。

安全版の出力（誰が考え，食べ，満腹になったかの並び）は 1000 の種で 1000 通りすべて異なっていた。同じプログラムでも動く順によって経過は毎回違い，それでも結果（全員が 3 回食べて満腹）は変わらない。これを手元で何度でも再現して見せられることが，教材としての実行系の役割である。

ただし，この実行系では「食事しています」は一通のメッセージの処理の中で終わり，時間の幅を持たない。そのため，隣り合わない 2 人が同時に食べている場面は観察できない。観察するには，食べ終わりを別のメッセージに分けて書く必要がある。

9.4　4 つの例題

アルゴリズムの定番 4 題を書き，種 1〜300 で実行した（表 6）。4 題とも 300 回すべて正しい答えを返した。

■ハノイの塔：返事を待つ再帰． ことのはには手続き呼び出しがない。そこで，「n 枚を移す」仕事を受けた移し係は，上の $n-1$ 枚を**子の移し係に頼み**，「済んだ」という返事を待ってから一番下の円盤を動かし，残りの $n-1$ 枚をもう一度子に頼む（図 3）。再帰呼び出しを，依頼と返事の組で書いたことになる。返事を待ってから次へ進むので，動く順をどう変えても出力は 1 通りであった。並列の言語で逐次の手順を正しく書く方法の例である。

■クイックソート：分けて任せる． 並べ係は，列の先頭を軸に取り，軸より小さい組とそれ以外の組に分け，二組をそれぞれ新しい並べ係に任せる。二つの返事は届く順が決まっていないので，依頼に「左」「右」の印を付けて送り，返事に印を付けて返してもらう。二つそろったら「小さい組・軸・大きい組」の順につなげて依頼主に返す。左右の係は並行に動くので途中の表示の並びは 56 通りに分かれたが，最後の結果はすべて同じであった。ユークリッドの互除法でも，二組の計算の途中経過が 202 通りに混ざり，答えは変わらなかった。

ハノイの塔（移し係の「済んだ」の受信節，文章形式）

移し係が「済んだ」というメッセージを受け取ったとき、
段階が 1 ならば、次のことを行う。
「円盤」という文字列と枚数と「を」という文字列と元と「から」という文字列と先と「へ移す」という文字列を表示し、段階を2に変更する。
空の配列を用意し、依頼と名付け、（枚数から1を引いた数）と経由と先と元と自分自身を依頼の末尾に追加し、子に「移せ」というメッセージと依頼を送る。
以上。
そうでなければ、依頼主に「済んだ」というメッセージを送る。

図 3　ハノイの塔。一度目の「済んだ」で一番下の円盤を動かして残りを頼み，二度目の「済んだ」で依頼主に返事をする。

■例題のために加えた機能． 当初の文法には，引き算・余り・要素数・範囲に式を書く繰り返し・要素への代入がなく，これらの例題を書けなかった。加えたのは計算の括弧形 3 種，「の要素数」，範囲の式，要素への代入である。文法は 85 規則・171 状態から 90 規則・185 状態になり，衝突は 0 件のままであった。例題を書く過程で，複数の値を一度に追加する文（「A と B を X に追加する」）の構造化形式に，内部の区切り文字が漏れる既存の不具合も見つかり，修正した。既存の見本だけでは通らない経路であり，例題は機能の追加と同時に検査の役も果たした。

10　考察

■自然さと一意性の折り合い． 一文方式は，日本語の文章としての自然さをある程度犠牲にしている。人が書く文章では「〜ならば、」の後ろに何文も続けることがあるが，ことのはでは「次のことを行う。」で囲まねばならない。その代わりに，読み方が衝突 0 の文法で保証され，構造化形式で目に見える。著者は，プログラムとして読む文章にはこの程度の約束事は受け入れられると考える。法律の条文が「次に掲げる」「各号」といった定型で範囲を示すのと同じ種類の約束事である。

■一文の長さ． 連用形でいくらでもつなげられるので，一文が長くなりうる。逆変換器は 3 節までで切る標準形を出す。何節までが読みやすいかは，学習者を対象に測る必要がある。

■照応．「その哲学者」の「その」は現在は読み飛ばしているだけで，指すものを確かめていない。日本語らしさの多くは照応と省略にあるが，それを入れると一意性の保証が文法の外に出る。今後，意味の段階で検査する形で取り入れたい。

■名前の切り出し． 名前を前もって正規表現で集める方式は単純で速いが，宣言の言い回しを少しでも崩すと名前が集まらない。宣言の形を文法から自動的に導く方式に置き換えるのが望ましい。

■誤りの位置． 表 4 の E2 のように，閉じ忘れは原因から離れた所で報告される。「次のことを行う。」の開始位置を覚えておき，閉じる語を待ったまま次の定義の頭に来たら，その位置を一緒に示すことで改善できる。

■生成 AI との共同設計． 本言語の初期の設計案は，対話型の生成 AI との議論の中で作った。その案には，日本語としては自然だが二通りに読める箇所がいくつもあった。一文方式は，その曖昧さを文法生成系の衝突として**数えられる形にして**一つずつ消す過程で得たものである。人の目で「読めるか」を判断するのではなく，道具に衝突を数えさせることが，自然言語風の言語を設計するうえで有効であった。

11　おわりに

日本語の文章として読めるアクター並列言語「ことのは」を設計し，実装した。一文方式によって，字下げも括弧も使わずに衝突 0 の LR(1) 文法を得た。また，文章形式と構造化形式の二つの表記を，構造を変えずに往復で

きるようにした。ブラウザ上の処理系で，5 人の哲学者のデッドロックと，その回避を，種を変えながら何度でも再現できる。4 つの例題は，手続き呼び出しのないアクター言語で，再帰（ハノイの塔）と分割統治（クイックソート）を書く方法を示した。

今後の課題は，照応の検査，名前の宣言の文法化，誤りの報告位置の改善，そして学習者による読みやすさの評価である。また，本言語で書いたプログラムを，著者らのアクター言語 AIPL [12] に変換して実機の並列環境で動かすことも計画している。

■付記． 本研究の言語の初期設計では対話型生成 AI（ChatGPT）を，処理系の実装と本稿の下書きでは Claude（Anthropic）を用いた。本文中の数値は，すべて公開している処理系を実際に動かして得たものである。

参考文献

[1] C. Hewitt, P. Bishop, R. Steiger: A Universal Modular ACTOR Formalism for Artificial Intelligence, *Proc. IJCAI*, pp. 235–245 (1973).

[2] G. Agha: *Actors: A Model of Concurrent Computation in Distributed Systems*, MIT Press (1986).

[3] J. Armstrong: *Programming Erlang: Software for a Concurrent World*, Pragmatic Bookshelf (2007).

[4] E. W. Dijkstra: Hierarchical Ordering of Sequential Processes, *Acta Informatica*, Vol. 1, No. 2, pp. 115–138 (1971).

[5] D. E. Knuth: On the Translation of Languages from Left to Right, *Information and Control*, Vol. 8, No. 6, pp. 607–639 (1965).

[6] P. J. Landin: The Next 700 Programming Languages, *Comm. ACM*, Vol. 9, No. 3, pp. 157–166 (1966).

[7] Free Software Foundation: *GNU Bison Manual*, version 3.8, `https://www.gnu.org/software/bison/manual/`.

[8] Z. Carter: Jison — an API for creating parsers in JavaScript, `https://github.com/zaach/jison`.

[9] クジラ飛行机: 日本語プログラミング言語「なでしこ」, `https://nadesi.com/`.

[10] 兼宗 進，中谷 多哉子，御手洗 理英，福井 眞吾，久野 靖: 学校教育用オブジェクト指向言語「ドリトル」の設計と実装, 情報処理学会論文誌：プログラミング, Vol. 42, No. SIG11 (PRO12), pp. 78–90 (2001).

[11] G. Nelson: Natural Language, Semantic Analysis and Interactive Fiction, `https://inform7.com/` (2006).

[12] 児玉 靖司: AIPL（Actor based Intelligent Parallel Language）紹介, `https://kodamay.org/aipl-intro/`.